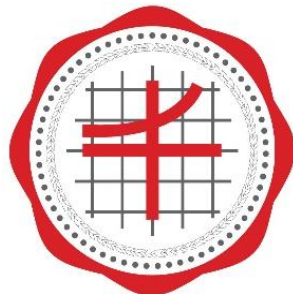


CPE 121 Mobile Application Developments

บทที่ 9

การสืบทอดและการอินเทอร์เฟส (Inheritance and Interface)



Faculty of Engineering

SRINAKHARINWIROT UNIVERSITY

Pramual Choorat, Ph.D.

Department of Computer Engineering

การสืบทอด

การสืบทอด (Inheritance) เป็นคุณสมบัติหนึ่งของ OOP ที่อนุญาตให้ผู้เขียนโปรแกรมสามารถสร้างคลาสใหม่โดยสืบทอดเอาคุณสมบัติต่าง ๆ มาจากคลาสที่มีอยู่แล้วมาใช้ก็เพื่อประหยัดเวลาในการสร้างคลาส คือ จะได้นำสิ่งที่มีอยู่แล้วใน คลาสเดิมมาใช้ได้เลย โดยที่ไม่ต้องเสียเวลาสร้างใหม่

เมื่อ class หนึ่งสืบทอดคุณสมบัติมาจากอีก class หนึ่ง

- เรียก class ที่ถูกสืบทอดคุณสมบัติไปว่า คลาสแม่ (super class/ base class/ parent class)
- เรียก class ที่สืบทอดคุณสมบัติมาว่า คลาสลูก (subclass/derived class/ child class)

การสืบทอดคลาสแม่ (Super class) จะทำการถ่ายทอดแอตทริบิวต์และเมธอดจากคลาสแม่ให้กับคลาสลูก (Sub class) คลาสลูกจึงสามารถทำการเรียกใช้งานแอตทริบิวต์และเมธอดที่สืบทอดมาได้เลย เสมือนถูกสร้างไว้ในคลาสของตัวเอง นอกจากนี้ คลาสลูกสามารถกำหนดแอตทริบิวต์และเมธอดเพื่อสร้างเพิ่มเติมเข้าไปในคลาสลูกได้ด้วย ดังนั้น คลาสลูกจะมีแอตทริบิวต์และเมธอดที่ได้รับถ่ายทอดมา และก็มีแอตทริบิวต์และเมธอดที่สร้างขึ้นมาเองเพิ่มเติม



คลาสแม่

คลาสแม่ (Super class) เป็นคลาสที่เป็นต้นแบบของแอตทริบิวต์และเมธอด จะอนุญาตให้คลาสอื่นสืบทอดแอตทริบิวต์และเมธอดไปได้ทั้งหมด ยกเว้นแอตทริบิวต์ที่มีระดับการเข้าถึงเป็น **private** และเมธอดที่เป็น **constructor** โดยระดับการเข้าถึงแอตทริบิวต์และเมธอดของคลาสแม่เป็น protected มีรูปแบบการใช้งาน ดังนี้

```
[modifier] class SuperClassName {  
    [AttributeName]  
    [MethodName]  
}
```

โดยที่

modifier

SuperClassName

AttributeName

MethodName

เป็นคีย์เวิร์ดที่กำหนดคุณสมบัติการเข้าถึงคลาส

เป็นชื่อคลาสแม่

เป็นส่วนของการประกาศแอตทริบิวต์

เป็นส่วนของการประกาศเมธอด



คลาสลูก

คลาสลูก (Sub class) เป็นคลาสที่สืบทอดแอตทริบิวต์และเมธอดจากคลาสแม่ คลาสลูกสามารถเรียกใช้แอตทริบิวต์และเมธอดจากคลาสแม่ได้ และคลาสลูกมีแอตทริบิวต์และเมธอดเพิ่มเติมเป็นของตัวเองได้ การระบุความสัมพันธ์ให้คลาสเป็นคลาสลูกต้องใช้คีย์เวิร์ด **extends** มีรูปแบบการใช้งาน ดังนี้

```
[modifier] class SubClassName extends SuperClassName {  
    [AttributeName]  
    [MethodName]  
}
```

โดยที่

modifier	เป็นคีย์เวิร์ดที่กำหนดคุณสมบัติการเข้าถึงคลาส
SubClassName	เป็นชื่อคลาสลูก
SuperClassName	เป็นชื่อคลาสแม่
AttributeName	เป็นส่วนของการประกาศแอตทริบิวต์
MethodName	เป็นส่วนของการประกาศเมธอด



คลาสแม่และคลาสลูก

ตัวอย่าง การสร้างคลาสแม่ – คลาสลูก

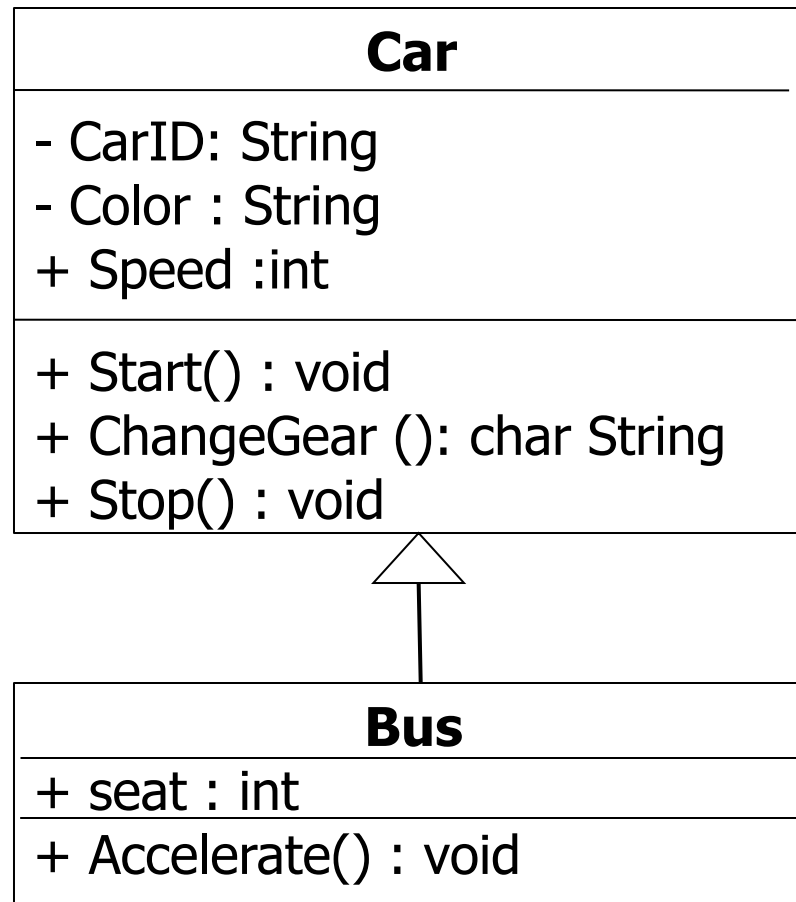
- คลาส **Car** เป็นคลาสรถทั่วไปที่สามารถ
 - สตาร์ทเครื่องได้ – **start()**
 - เปลี่ยนเกียร์ได้ – **ChangeGear()**
 - ดับเครื่องได้ – **stop()**
- คลาส **Bus** เป็นคลาสรถบัสที่สามารถ
 - สตาร์ทเครื่องได้
 - เปลี่ยนเกียร์ได้
 - ดับเครื่องได้
 - มีผู้โดยสาร - **Seat**

ดังนั้น คลาส **Car** เป็นคลาสแม่ และ คลาส **Bus** เป็นคลาสลูกที่สืบทอดคุณสมบัติจากคลาส **Car**



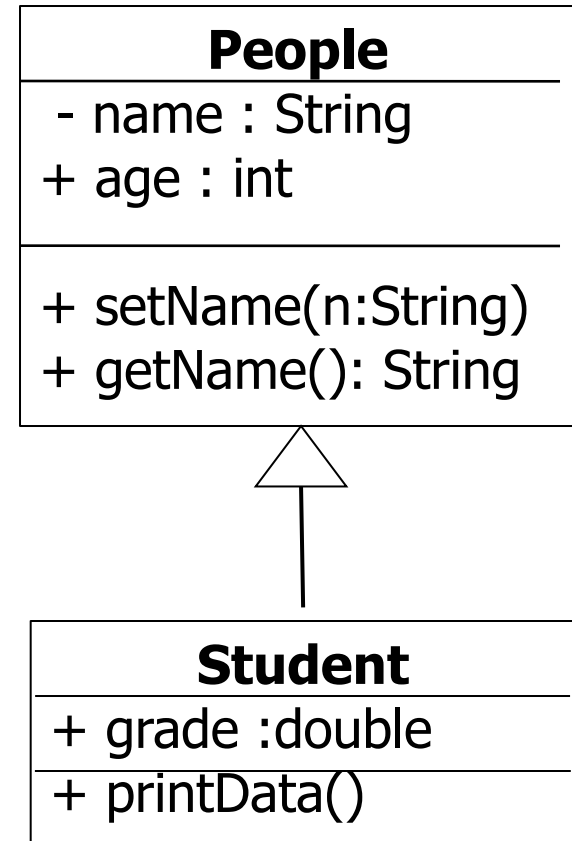
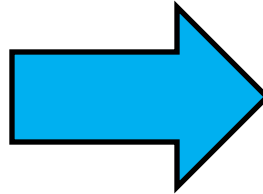
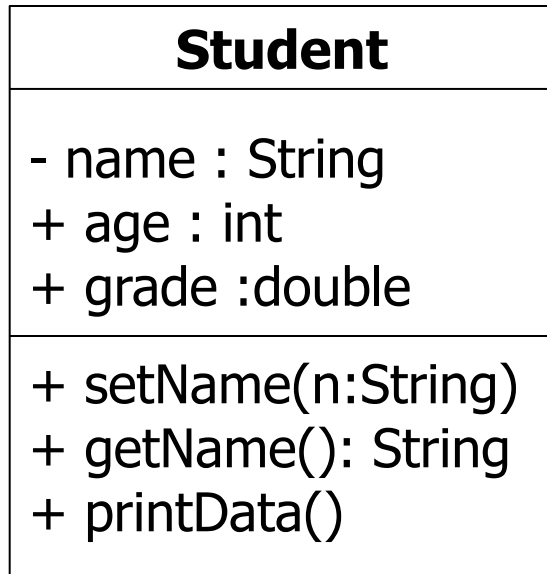
คลาสแม่และคลาสลูก

ตัวอย่าง การสร้างคลาสแม่ – คลาสลูก



คลาสแม่และคลาสลูก

ตัวอย่าง การสร้างคลาสแม่ – คลาสลูก



ตัวอย่าง โปรแกรมคลาสแม่และคลาสลูก

```
public class Student1 extends People{
    public static void main (String [] args){
        Student1 s = new Student1();
        s.setName("CPE 121");
        System.out.println("Name : " + s.getName());
        s.printData();
    }
    double grade;
    public Student1() {
        grade = 3.06;
        age = 6;
    }
    public void printData() {
        System.out.println("Age : " + age);
        System.out.println("Grade : " + grade);
    }
}
```

```
public class People{
    private String name;
    public int age;
    public void setName(String n){
        name = n;
    }
    public String getName() {
        return name;
    }
}
```

> run **Student1**

Name : CPE 222

Age : 6

Grade : 3.06

การพ้องรูป

การพ้องรูป (Polymorphism) เป็นคุณสมบัติประการหนึ่งของการเขียนโปรแกรมเชิงวัตถุที่เอื้ออำนวยประโยชน์ต่อการนำคลาสกลับมาใช้งานอีก เป็นกลไกสำคัญสำหรับการเขียนโปรแกรมเชิงวัตถุ ที่เป็นผลพวงของการสืบทอดคุณสมบัติการพ้องรูป มีรูปแบบในการใช้งานด้วยกัน 2 แบบ

1. การพ้องรูปแบบสถิต (Static polymorphism) : เป็นสถานะที่เมธอดมีหลายรูปแบบ Function overloading, Operator overloading

2. การพ้องรูปแบบไดนามิก (Dynamic polymorphism) : เป็นวิธีการกำหนดรูปแบบการกระทำที่เหมือนกันแต่ได้ผลที่แตกต่างกัน



โอเวอร์โหลด

Overloading Method

- เป็นเมธอดที่มีคุณลักษณะที่มีได้หลายรูปแบบ โดยใช้ชื่อเมธอดเดียวกันมากกว่า 1 เมธอด เพื่อทำงานในแบบเดียวกัน
- สิ่งที่แตกต่างกัน คือ ชนิดข้อมูลของผลลัพธ์หรือมีจำนวนและชนิดข้อมูลของอาร์กิวเมนต์ที่ใช้ในการรับข้อมูลแตกต่างกัน

Overload Constructor

- เป็นคอนสตรัคเตอร์ ที่มีมากกว่าหนึ่งตัวในคลาสเดียวกัน มีหลักการเดียวกับ Overloading Method โดยที่จำนวนหรือชนิดข้อมูลของอาร์กิวเมนต์ที่ส่งไปในแต่ละคอนสตรัคเตอร์ต้องแตกต่างกัน
- คอมไพเลอร์จะเรียกใช้คอนสตรัคเตอร์ ตัวใดให้ทำงานนั้นจะพิจารณาจากจำนวนหรือชนิดข้อมูลของอาร์กิวเมนต์ที่สอดคล้องกัน ทำให้กำหนดค่าเริ่มต้นให้กับออบเจกต์ได้หลายรูปแบบ



โอเวอร์โหลด : การใช้คีย์เวิร์ด **super**

- **คอนสตรัคเตอร์**เป็นสิ่งที่คลาสลูกไม่สามารถสืบทอดจากคลาสแม่ได้ แต่คลาสลูกสามารถใช้งาน คอนสตรัคเตอร์ของคลาสแม่ได้ โดยใช้คีย์เวิร์ด **super** จึงสามารถทำโอเวอร์โหลดคอนสตรัคเตอร์ในคลาสแม่ได้ตามปกติ
- คลาสลูกที่ต้องการเรียกใช้คอนสตรัคเตอร์ในคลาสแม่จึงใช้เมธอด **super()** เรียกใช้เป็นคำสั่งแรกสุดของคอนสตรัคเตอร์และมีรูปแบบเหมือนกับการเรียกใช้ด้วยคำสั่ง **new** โดยรูปแบบการใช้จะมีลักษณะดังนี้

super(), **super(parameters)** ;

- นอกจากนี้ **super** ยังเป็น keyword ที่สามารถใช้อ้างถึงถึง attribute และ method ของ superclass ของ class นั้น ๆ ได้ โดยมีรูปแบบการใช้งานดังนี้

super.attributeName;

super.methodName(parameters);



ตัวอย่าง โปรแกรม Overload Constructor

```
import java.util.Scanner;
class Employee0{
    protected float rate = 300.0f, work = 10.0f, pay;
    public Employee0(int h) {
        pay = h*rate/work;
    }
    public Employee0(int h, float r) {
        pay = h*r/work;
    }
    public Employee0(int h, float r, float b) {
        this(h,r);
        if (h>100)
            pay += b;
    }
}
```

ตัวอย่าง โปรแกรม Overload Constructor

```
class daily_emp extends Employee 0{  
    public daily_emp (int h,float r) {  
        super(h,r);  
    }  
}
```

```
class monthly_emp extends Employee 0{  
    public monthly_emp (int h, float r, float b) {  
        super(h,r,b);  
    }  
}
```



ตัวอย่าง โปรแกรม Overload Constructor

```
import java.util.Scanner;
public class InheritOverLoadTest {
    public static void main(String[] args) {
        float rate;
        Scanner scan = new Scanner(System.in);
        System.out.print("Enter total OT hour : ");
        int hour = scan.nextInt();
        Employee0 emp1 = new Employee0(hour);
        System.out.println("\nEmployee\nTotal OT Pay = (" + emp1.rate + " * " +
hour + ") = " + emp1.pay + " BAHT");
        System.out.print("\nDaily Employee\nRate per Day = ");
        rate = scan.nextFloat();
        daily_emp emp2 = new daily_emp(hour,rate);
        System.out.println("Total OT Pay = (" + emp2.rate + " * " + hour + ") = " +
emp2.pay + " BAHT");
    }
}
```



ตัวอย่าง โปรแกรม Overload Constructor

```
System.out.print("\nMonthly Employee\n\nEnter Salary = ");
rate = scan.nextFloat();
System.out.print("Enter Bonus = ");
float bonus = scan.nextFloat();
monthly_emp emp3 = new monthly_emp(hour,rate/30,bonus);
System.out.println("Total OT Pay = (" + emp3.rate + " * " + hour + ") + " +
bonus + " = " + emp3.pay + " BAHT");
}
}
```

> run InheritOverLoadTest

Enter total OT hour : 20

"Employee"

Total OT Pay = (300.0 * 20) = 600.0 BAHT

"Daily Employee"

Rate per Day = 350

Total OT Pay = (300.0 * 20) = 700.0 BAHT

"Monthly Employee"

Enter Salary = 9000

Enter Bonus = 600

Total OT Pay = (300.0 * 20) + 600.0 = 600.0 BAHT

โอเวอร์โหลด : ตัวอย่างการใช้คีย์เวิร์ด **super**

```
public class TestDog{  
    public static void main (String args[]){  
        Dog b = new Dog();  
        b.move();  
    }  
}
```

> **run TestDog**

Animals can move

Dogs can walk and run

```
class Animal{  
    public void move()  
        System.out.println("Animals can move");  
}
```

```
class Dog extends Animal{  
    public void move()  
        super.move();  
        System.out.println("Dogs can walk and run");  
}
```



โอเวอร์ไรด์

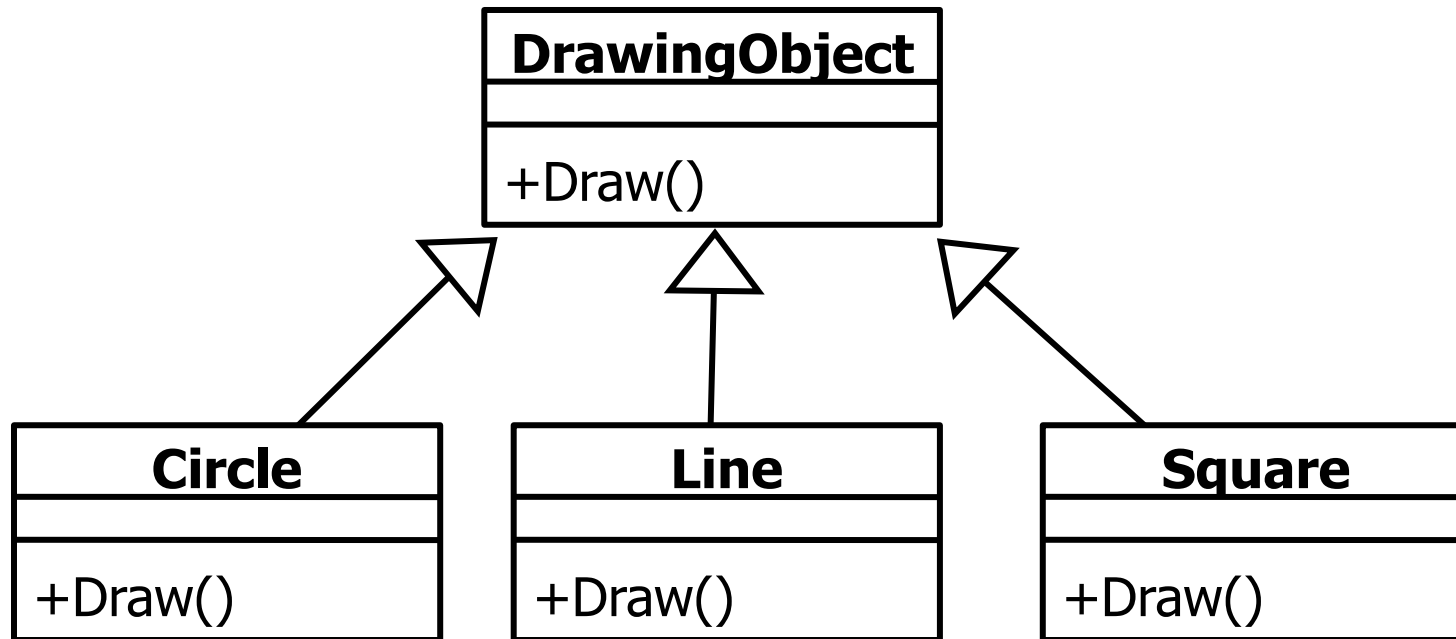
โอเวอร์ไรด์ (Overriding) เป็นการเปลี่ยนแปลงการทำงานของเมธอดในคลาสลูกที่สืบทอดมาจากคลาสแม่ เมธอดในคลาสลูกจะมีชื่อเมธอด, ชนิดข้อมูลที่คืนค่า, จำนวนและชนิดข้อมูลของอาร์กิวเมนต์ที่เหมือนกับคลาสแม่ เราสามารถพัฒนาเมธอดให้มีการทำงานในเรื่องเดียวกัน แต่แตกต่างกันในรายละเอียดของการทำงาน เช่น

- การคำนวณหาพื้นที่ของรูปทรงเรขาคณิตต่าง ๆ ที่มีสูตรคำนวณที่แตกต่างกัน
- การคำนวณค่าแรงของพนักงานที่มีวิธีคำนวณที่แตกต่างกันไปในแต่ละตำแหน่ง

ประโยชน์ของการโอเวอร์ไรด์ คือจะช่วยให้ คลาสลูกสามารถแก้ไขหรือเพิ่มเติมการทำงานบางอย่างของเมธอดที่ได้รับการถ่ายทอดมาจาก คลาสแม่



โอเวอร์โหลด



โอเวอร์โหลด

```
public String getName(){  
    // คำสั่งต่าง ๆ  
    String name="testName"  
    return name  
}
```

เมธอดจากคลาสแม่

```
public int getName(){  
    int x=1;  
    return x;  
}
```

```
public String getName(){  
    int y=0;  
    return "name is "+y  
}
```

```
public String getName(int x){  
    return "name is +";  
}
```

เมธอดจากคลาสลูก

โอเวอร์โหลด

```
public void setName(String x,int y){  
    // คำสั่งต่าง ๆ  
}
```

เมธอดจากคลาสแม่

```
public String setName(String x,int y){  
    return x  
}
```

```
public void setName(String y,int x){  
    //คำสั่งต่าง ๆ  
}
```

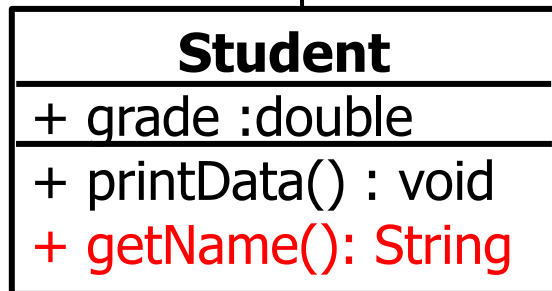
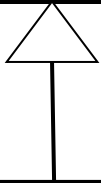
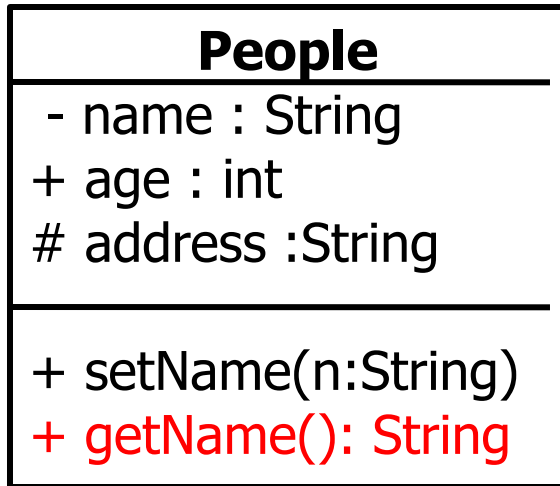
```
public void setName(int i,String j){  
    //คำสั่งต่าง ๆ  
}
```

```
public void setName(String x){  
    //คำสั่งต่าง ๆ  
}
```

```
public void setName(){  
    //คำสั่งต่าง ๆ  
}
```

เมธอดจากคลาสลูก

โอเวอร์โหลด



```
public class Student extends People
{ double grade;
  public void printData() {
    System.out.println("Age = " + age);
    System.out.println("Address = " + address);
    System.out.println ("Grade = " + grade);
  }
  public String getName() {
    return "getName() from subClass "
  }
}
```



ตัวอย่าง การโอเวอร์โหลด 1

```
class Animal{  
    public void move() {  
        System.out.println("Animals can move");  
    }  
}
```

```
class Dog extends Animal{  
    public void move() {  
        System.out.println("Dogs can walk and run");  
    }  
    public void bark() {  
        System.out.println("Dogs can bark");  
    }  
}
```

```
public class TestDog{  
    public static void main (String args[]){  
        Animal a = new Animal();  
        Dog b = new Dog();  
        a.move();  
        b.move();  
        b.bark();  
    }  
}
```

> run **TestDog**

Animals can move

Animals can move

Dogs can walk and run

Dogs can bark



ตัวอย่าง การโอเวอร์โหลด 2

```
public class Manager1 extends EmployeeTest{
    public static void main (String [] args){
        EmployeeTest emp = new EmployeeTest();
        System.out.println("getInformation method from EmployeeTest class");
        System.out.println(emp.getInformation());

        Manager1 mgr = new Manager1();
        System.out.println("getInformation method from Manager1 class");
        System.out.println(mgr.getInformation());
    }
    protected String dept = "Data Network";
    public String getInformation() {
        return "Name : " + name + "\n" + "Salary: " + salary + "\n" + "Manager of : " +
dept;
    }
}
```



ตัวอย่าง การโอเวอร์โหลด 2

```
public class EmployeeTest{
    protected String name = new String("Somsak");
    public double salary;
    public EmployeeTest(){
        salary = 20000.00;
    }
    public String getInformation(){
        return "Name : " + name+ "\n" + "Salary: " + salary + "\n";
    }
}
```

> run Manager1

getInformation method from EmployeeTest class

Name : Somsak

Salary: 20000.0

getInformation method from Manager1 class

Name : Somsak

Salary: 20000.0

Manager of :Data Network

ความต่างระหว่างโอเวอร์ไรด์ิงและโอเวอร์โหลด

```
public class TestOverride{
    public static void main (String args[]){
        A a = new A();
        a.p(10);
        a.p(10.0);
    }
}
class B{
    public void p(double i){
        System.out.println(i =2);
    }
}
class A extends B{
    @Override
    public void p(double i){
        System.out.println(i);
    }
}
```

> run TestOverride

10.0

10.0



ความต่างระหว่างโอเวอร์โหลดิ่งและโอเวอร์โหลด

```
public class TestOverloads{
    public static void main (String args[]){
        A a = new A();
        a.p(10);
        a.p(10.0);
    }
}
class B{
    public void p(double i){
        System.out.println(i =2);
    }
}
class A extends B{
    public void p(int i){
        System.out.println(i);
    }
}
```

> run TestOverloads

10

2.0



non-access modifier : static

static เป็นคีย์เวิร์ดที่ใช้กำหนดไว้หน้าเมธอดหรือตัวแปรเพื่อให้เมธอดหรือตัวแปรนั้นเป็นแบบ static

การกำหนดให้เมธอดเป็น static

การกำหนดให้เมธอดเป็น static (static method) จะทำให้เราสามารถเรียกเมธอดนั้นให้ทำงานได้โดยไม่ต้องสร้างออบเจกต์ของคลาสขึ้นมาด้วยคีย์เวิร์ด new

```
public class TestStaticMethod{  
    public static void main (String [] args){  
        System.out.println("number = " + Calculate.cal(3));  
    }  
}  
class Calculate{  
    public static int cal(int number){  
        return number+3;  
    }  
}
```

> run TestStaticMethod
number = 6

non-access modifier : final

final เป็นคีย์เวิร์ดที่ใช้หน้าหน้าตัวแปร, เมธอด และคลาส ทำให้มีคุณสมบัติดังนี้

- ตัวแปรเป็นค่าคงที่ที่ไม่สามารถเปลี่ยนแปลงค่าได้
- เมธอดไม่สามารถถูก **override** ได้
- คลาสไม่สามารถถ่ายทอดคุณสมบัติได้ คือไม่สามารถเป็นคลาสแม่ได้



non-access modifier : final

```
public final class TestFinal1{  
    public void print() {  
        System.out.println("TestFinal1 : print() : Static class");  
    }  
}
```

```
public class TestFinal2{  
    public void print() {  
        System.out.println("TestFinal2 : print() : Non-static class");  
        System.out.println("TestFinal2 : print() : Non-static method");  
    }  
    public final void printStatic() {  
        System.out.println("TestFinal2 : printStatic() : Non-static class");  
        System.out.println("TestFinal2 : printStatic() : Static method");  
    }  
}
```



non-access modifier : final

```
public class TestFinal extends TestFinal1{
//public class TestFinal extends TestFinal2{
//  public void print(){
//    System.out.println("TestFinal : print() : Non-static class");
//    System.out.println("TestFinal : print() : Non-static method");
//    System.out.println("TestFinal : print() : TestFinal");
//  }
//  public static void printStatic(){
//    System.out.println("TestFinal : printStatic() : Non-static class");
//    System.out.println("TestFinal : printStatic() : Static method");
//    System.out.println("TestFinal : printStatic() : TestFinal");
//  }
  public static void main (String [] args){
    TestFinal tfc = new TestFinal();
    tfc.print();
    tfc.printStatic();
  }
}
```



non-access modifier : final

```
//public class TestFinal extends TestFinal1{
public class TestFinal extends TestFinal2{
    public void print(){
        System.out.println("TestFinal : print() : Non-static class");
        System.out.println("TestFinal : print() : Non-static method");
        System.out.println("TestFinal : print() : TestFinal");
    }
    public static void printStatic(){
        System.out.println("TestFinal : printStatic() : Non-static class");
        System.out.println("TestFinal : printStatic() : Static method");
        System.out.println("TestFinal : printStatic() : TestFinal");
    }
    public static void main (String [] args){
        TestFinal tfc = new TestFinal();
        tfc.print();
        tfc.printStatic();
    }
}
```



non-access modifier : abstract

abstract เป็นคีย์เวิร์ดที่ใช้กำหนดเมธอดหรือคลาสเพื่อให้เมธอดหรือคลาสนั้นเป็นแบบ abstract

การกำหนดให้เมธอดเป็น **abstract**

เมธอดที่เป็น abstract (abstract method) คือ เมธอดว่างที่ยังไม่มีการกำหนดรายละเอียดการทำงานใดๆลงไป ในเมธอด ซึ่งในภายหลังรายละเอียดการทำงานของเมธอดเหล่านี้จะ**ถูกกำหนดโดยคลาสลูก**ที่ทำการสืบทอดคุณสมบัติไปจากคลาสของ abstract method

abstract method มีประโยชน์อย่างมาก เพราะบางครั้งรายละเอียดการทำงาน of คลาสลูกแต่ละคลาสไม่เหมือนกัน ดังนั้นแทนที่จะต้องกำหนดเมธอดขึ้นมามากมายเพื่อรองรับการทำงาน of คลาสลูกแต่ละคลาส ก็เพียงแค่กำหนดเมธอดว่างขึ้นมาเป็นโครงสร้างไว้ภายในคลาสแม่ก่อน แล้วให้คลาสลูกที่สืบทอดคุณสมบัติไปจากคลาสแม่ไปกำหนดรายละเอียดการทำงาน of เมธอดเองตามความต้องการภายหลัง



non-access modifier : abstract

การกำหนดให้คลาสเป็น **abstract**

หากคลาสใดประกอบไปด้วย**เมธอดที่เป็น abstract** แม้แต่เพียงคลาสเดียว จำเป็นจะต้อง**ประกาศคลาสนั้นให้เป็น abstract** (abstract class) ด้วย

หากคลาสใด**สืบทอดคุณสมบัติ**ไปจาก abstract class แล้ว **คลาสนั้นจะต้องทำการระบุเมธอดทุกเมธอดที่เป็น abstract method** ใน abstract class ไว้ด้วยเสมอ คือไม่ว่าจะต้องการกำหนดรายละเอียดการทำงานให้กับ abstract method ที่ได้รับถ่ายทอดมาจาก abstract class หรือไม่ก็ตาม ก็จำเป็นต้องระบุ abstract method ทุกเมธอดไว้ในคลาสลูก



non-access modifier : abstract

```
class MyTestAbstract extends TestAbstract{
    public static void main (String [] args){
        MyTestAbstract m = new MyTestAbstract();
        m.a1();
        m.a2();
        m.a3();
    }
    public void a1(){
        System.out.println("Test A1");
    }
    public void a2(){ }
}
abstract class TestAbstract{
    public abstract void a1();
    public abstract void a2();
    public void a3(){
        System.out.println("Test A3");
    }
}
```

ตัวอย่าง การใช้ abstract class abstract method

```
public abstract class MyEmployee{  
    public abstract String calSalary(int dataForCalSalary);  
    public abstract String getFirstEmpid();  
}
```

```
public class MyHourlyEmployee extends MyEmployee{  
    private double salary;  
    public String calSalary(int hourOfWork){  
        salary = hourOfWork*35;  
        return "Salary " + salary;  
    }  
    public String getFirstEmpid(){  
        return "First Emp Id : 9\n";  
    }  
}
```



ตัวอย่าง การใช้ abstract class abstract method

```
public class MyManager extends MyEmployee{  
    private double salary;  
    public String calSalary(int managerType){  
        if(managerType == 1){salary = 30000;}  
        else if(managerType == 2){salary = 35000;}  
        else if(managerType == 3){salary = 50000;}  
        return "Salary " + salary;  
    }  
    public String getFirstEmpid()  
    {  
        return "First Emp Id : 8\n";  
    }  
}
```



ตัวอย่าง การใช้ abstract class abstract method

```
public class TestAbstractClass{
    public static void main(String[] args){
        MyHourlyEmployee h = new MyHourlyEmployee();
        MyGeneralEmployee g = new MyGeneralEmployee();
        MyManager m = new MyManager();
        System.out.println("Hourly Employee");
        System.out.println(h.calSalary(180));
        System.out.println(h.getFirstEmpid());
        System.out.println("General Employee");
        System.out.println(g.calSalary(113));
        System.out.println(g.getFirstEmpid());
        System.out.println("Manager");
        System.out.println(m.calSalary(3));
        System.out.println(m.getFirstEmpid());
    }
}
```

> run TestAbstractClass

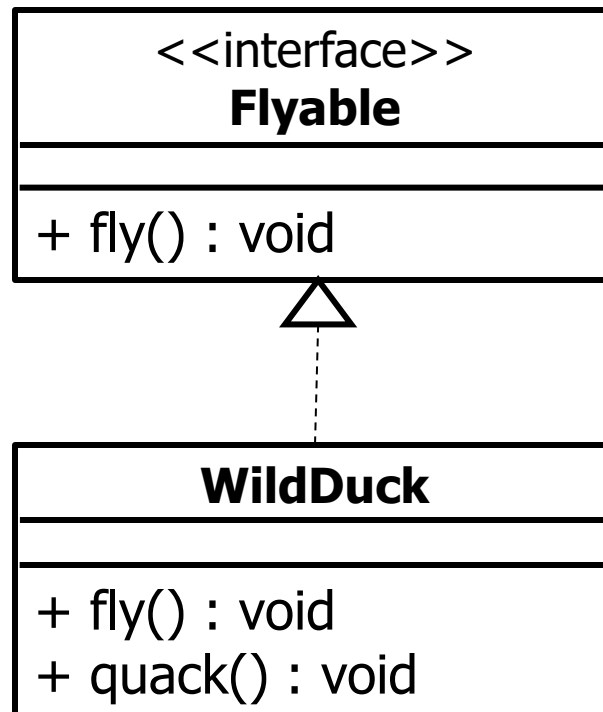
Hourly Employee
Salary 6300.0
First Emp Id : 9

General Employee
Salary 15000.0
First Emp Id : 1

Manager
Salary 50000.0
First Emp Id : 8

อินเทอร์เฟส

อินเทอร์เฟส (Interface) มีหลักการคล้ายๆ กับ abstract class คือเราสร้างอินเทอร์เฟสขึ้นเพื่อกำหนดโครงสร้างของเมธอดที่จำเป็นต้องใช้งานขึ้นมา แต่ยังไม่กำหนดรายละเอียดการทำงานใดๆ ลงไปในเมธอด เมธอดต่างๆ ในอินเทอร์เฟสจึงเป็นเมธอดที่ว่างเปล่า ซึ่งรายละเอียดการทำงานของเมธอดเหล่านี้จะถูกกำหนดโดยคลาสที่เรียกใช้งานอินเทอร์เฟสนั้นๆ อีกที



ความต่างระหว่าง Interface กับ Abstract class

เมธอดบางเมธอดใน abstract class จะถูกประกาศเป็น **abstract** แต่เมธอดในอินเทอร์เฟซเราไม่ต้องประกาศเป็น **abstract** เพราะทุกเมธอดในอินเทอร์เฟซจะเป็น **abstract** โดยปริยายอยู่แล้ว

คลาสที่จะเรียกใช้งาน abstract method ใน abstract class จะต้อง **สืบทอดคุณสมบัติไปจาก abstract class** นั้นแล้วจึงทำการสร้างเมธอดของตนเองขึ้นมาให้มีชื่อเดียวกับ abstract method ใน abstract class โดยกำหนดรายละเอียดการทำงานให้กับเมธอดตามต้องการ คือ จะต้องทำการ **override abstract method** ใน **abstract class** นั้นเอง แต่คลาสที่จะเรียกใช้งานเมธอดในอินเทอร์เฟซไม่จำเป็นต้องมีความสัมพันธ์ใดๆกับอินเทอร์เฟซทั้งสิ้น



การใช้งานอินเทอร์เฟส

การประกาศคลาสอินเทอร์เฟส

```
public interface Entertainer{  
    public void sing();  
    public void dance();  
}
```

การเรียกใช้งานอินเทอร์เฟส

```
public class Singer implements Entertainer{  
public class Student implements Entertainer{
```

การเรียกใช้งานอินเทอร์เฟสที่มากกว่า 1 อินเทอร์เฟส

```
public class Singer implements Entertainer, Musician{
```



การใช้งานอินเทอร์เฟส

คลาสที่ทำการอิมพลีเมนต์อินเทอร์เฟสใด จะต้องระบุเมธอดที่มีอยู่ในอินเทอร์เฟสนั้นให้ครบทุกเมธอด คือ ไม่ว่าจะต้องกำหนดรายละเอียดการทำงานให้กับเมธอดในอินเทอร์เฟสหรือไม่ก็ตาม จะต้องกำหนดเมธอดทุกเมธอดไว้ในคลาสด้วยเสมอ

```
public interface Entertainer{  
    public void sing();  
    public void dance();  
}
```

```
public class Student implements Entertainer{  
    public void sing() {  
        System.out.println("###Student can sing###");  
    }  
    public void dance() {  
        System.out.println("###Student can dance###");  
    }  
}
```

การใช้งานอินเทอร์เฟส

```
public interface Musician{  
    public void playGuitar();  
    public void playPiano();  
}
```

```
public class Singer implements Entertainer, Musician{  
    public void sing() {  
        System.out.println("Singer can sing");  
    }  
    public void dance() {  
        System.out.println("Singer can dance");  
    }  
    public void playGuitar() {  
        System.out.println("Singer can play Guitar");  
    }  
    // public void playPiano(){  
    //     System.out.println("Singer can play Piano");  
    // }  
}
```

การใช้งานอินเทอร์เฟส

```
class TestSimpleInterface{  
    public static void main(String [] args){  
        Singer singer = new Singer();  
        Student student = new Student();  
        singer.sing();  
        singer.dance();  
        singer.playGuitar();  
        // singer.playPiano();  
        student.sing();  
        student.dance();  
    }  
}
```

